# 《MVC 原始文稿》中文全译

原作者：Trygve Reenskaug

汇编日期：2007 年 2 月 12 日

译文依据：The Original MVC Reports，包括 1979 年 5 月 12 日和 12 月 10 日的两份原始文稿。

说明：正文按原文次序翻译；类名、消息名、Smalltalk 标识符保持原样。插图均收录在本译本后附的英文原页中。

## 编者序言：MVC 原始文稿（2007 年）

Trygve Reenskaug，Oslo大学信息学系

1978 至 1979 年间，我在Xerox Palo Alto研究中心（Xerox PARC）担任访问科学家时，完成了 MVC 的第一个实现，并写下了最初的 MVC 文稿。MVC 的产生，是为解决这样一个普遍问题：让用户能够从多个不同的视角查看自己的信息，并掌握对这些信息的控制权。这在当时看来是一个自然的解决方案。

MVC 后来引起了出人意料的广泛兴趣。有些文章甚至采用了被扭曲的变体，反而让计算机去控制用户——这与最初的目标正好相反。

MVC 最初被构想为一个通用方案，用于解决用户如何控制大量而复杂的数据集合的问题。其中最困难的部分，是为不同的架构组成部分找到恰当的名称。最初的一组名称是 Model–View–Editor（模型—视图—Editor）。我在 1979 年 5 月 12 日的第一份笔记《THING–MODEL–VIEW–EDITOR：来自规划系统的一个例子》中描述了它们。该笔记的扫描件附在后面。

经过长时间讨论，特别是与 Adele Goldberg 的讨论，我最终采用了 Model–View–Controller（模型—视图—控制器）这组术语。它们在 1979 年 12 月 10 日的第二份笔记《MODELS–VIEWS–CONTROLLERS》中得到描述。该笔记的扫描件也收在本文末尾。

在我离开 Xerox PARC 之后，Jim Althoff 等人为 Smalltalk-80 类库实现了一个 MVC 版本。我并未参与那项工作。

Oslo，2007 年 2 月 12 日

Trygve Reenskaug

# 第一份文稿（1979 年 5 月 12 日）

## THING–MODEL–VIEW–EDITOR：来自规划系统的一个例子

收件人：LRG

发件人：Trygve Reenskaug

存档位置：`[IVY]<Reenskaug>SMALL>TERMIN0LOGY2.DOC`

日期：1979 年 5 月 12 日

这份笔记的目的，是通过一组相互关联的例子，探索 Thing–Model–View–Editor（事物—模型—视图—Editor）这些概念。所有例子都取自我的规划系统，用于说明上述四个概念。所有例子都已实现，不过其实际类结构尚未达到这里描述的那样清晰。这些概念对应于《DynaBook 需求笔记》（`[Ivy]<Reenskaug>DynaBook.doc`）所提出的现实世界—模型—视图—工具。

## 1. 事物（THING）

### 术语说明

事物，就是用户感兴趣的某个东西。它可以是具体的，比如住或集成电路；也可以是抽象的，比如一个新想法或对论文的看法。它可以是一个整体，比如计算机；也可以是一个部分，比如电路元件。

### 示例：一个大型项目

这里的事物是一个大型项目，例如建造一座主要bridge、一座电或一座海上石油生产平台。

这样的项目包含非常复杂的任务，存在数量极多且相互依赖的细节。项目负责人必须追踪这些细节及其依赖关系，才能理解实际发生或设想中可能发生的各种情况所带来的后果。

为了管理大型项目，人们会使用许多不同的抽象方式。每一种抽象都会突出整个项目的某些特定方面，并可以单独使用或与其他抽象一起使用，以便控制这些方面。例如：如何理解材料需求、如何估算成本，以及各种预算与会计理论。

一个特别有用的抽象方式，是活动网状结构（Activity Network），也就是 PERT 图。它把“要做什么”“由哪一人来做”“什么时候做”联系起来。

在网状结构这种抽象中，项目里的每一项待办任务都会被映射为一个简单元素，称作活动（Activity）。基本上，一个活动由它的持续时间和前驱活动来描述。持续时间指完成相应任务所需的时间；前驱活动是指当前活动开始前必须完成的活动。

这种简单的抽象通常会通过多种方式扩展。根据活动的前驱关系，很容易找出每个活动的后继活动。网状结构中的起始活动没有前驱，终止活动没有后继。我们还可以为每项活动添加其他信息，例如所需资源，以及与活动有关的成本和现金流。

## 2. 模型（MODEL）

### 定义

模型是计算机系统中以数据形式存在的、对某种抽象的主动表示。

### 说明

如前所述，对同一事物通常可以采用多种抽象方式。因此，为一个特定事物建立几个同时存在的模型往往很有用。另一种理解方式是：把项目看作拥有一个大型模型，而该模型又被划分为若干子模型。

在计算机中，模型由数据集合及处理这些数据所需的方法共同表示。

理想情况下，所有模型都应完全一致。但现实中无法达到这一理想，因为那需要大量到难以承受的管理程序，并带来过度的刻板。因此应当允许一定程度的不一致，目标是在不为无关紧要的细节付出过多心力的前提下，让所有模型合起来仍能相当准确地表示项目。

在我们的例子中，一个给定的网状结构模型，在 Smalltalk 系统中由 `NetworkModel` 类的实例表示。网状结构中的每个活动由 `Activity` 类的实例表示。

`NetworkModel` 定义的字段之一是 `activities`：它是一个 `Dictionary`，把活动名称（`UniqueString`）映射到 `Activity` 的实例。

`Activity` 类定义的字段中，有三个分别是 `network`、`predecessors` 和 `successors`。`network` 保存指向有关 `NetworkModel` 实例的指针；`predecessors` 和 `successors` 则是 `Vector`，其中存放指向相应 `Activity` 实例的指针。

由此，我们的模型（即项目网状结构模型在 Smalltalk 中的表示）具有如下总体结构：一个 `NetworkModel` 关联多个 `Activity`；活动又通过前驱与后继关系相互联系。（见原文第 3 页结构图。）

在这个总体框架下，我们可以继续加入与整个网状结构或某个活动有关的项目信息。例如，可为网状结构增加 `plannedStart`（计划开始）和 `plannedFinish`（计划完成）字段；为 `Activity` 增加 `duration`（持续时间）、`earlyStart`（最早开始）、`lateStart`（最迟开始）和 `resourceRequirements`（资源需求）字段。

我们还可以增加最迟开始、最迟完成之类的字段，但也可以决定只在需要时计算这些量（依据持续时间以及最早或最迟开始时间）。

请注意，网状结构模型及其为每个活动设置的子模型，没有包含有关如何在屏幕上显示信息的内容。它是对抽象项目模型的纯正表示。

## 3. 视图（VIEW）

### 定义

对于任意给定的模型，都有一个或多个与之关联的视图。每个视图都能够在屏幕或纸质输出上展示该模型的一种或多种图形表示。视图也能够对模型执行与该视图合理相关的操作。

### 说明

如果所有视图的实现都建立在 Form–Path–Image（形状—路径—图像）这些概念之上，实现工作将得到简化。

### 示例 1：网状结构列表

这个视图属于一个超级网状结构，即多个网状结构组成的集合。因此，它稍微超出了此前讨论的模型范围，但为了完整性仍将它列在这里。网状结构列表的外观见原文左侧插图。

网状结构列表是 `NetworkList` 类的实例；`NetworkList` 是 `ListView` 的子类。

`ListView` 有几个字段，用于记住自己的显示区域、文本项列表及列表中可能存在的选择项。它能在自己的显示区域内将列表画到屏幕上，并能响应要求它滚动的消息。它理解这样的消息：给定显示区域中的一个点，返回该位置对应的列表项；也理解要求它选择某一项的消息。

因此，`ListView` 有些类似当前系统的 `ListPane`，但它不是 `Window` 的子类，也不能独立参与调度。它必须依赖 `Editor` 告知显示区域的位置，并布置滚动。

一种可能的选择流程是：`Editor` 响应 `redbug` 操作，向 `ListView` 问问指向了哪一项，然后要求该 `ListView` 和其他视图一起选择这一项。将用户输入界面与实际命令执行分开，能为 `Editor` 带来很大的灵活性。

`NetworkList` 建立在 `ListView` 的通用功能之上。它必须知道所处理的列表是网状结构列表，以及如何获得这个列表。此外，它还必须能够执行与该视图有关的命令。这里仅讨论两个：返回被选中网状结构的名称，以及通知 `Editor` 去修改某个给定的网状结构。

### 示例 2：网状结构中的活动列表

网状结构内的活动列表是 `ActivityList` 类的实例。它也是 `ListView` 的子类（通用类的说明见上文）。

`ActivityList` 的实例必须知道自己属于哪个网状结构，以及如何获得该网状结构中的全部活动列表。除此之外，它不需要特殊命令，只需要 `ListView` 提供的通用选择机制。

## 示例 3：活动属性视图

这是某项活动全部属性的文本表示。它是 `ActivityText` 类的实例，`ActivityText` 是 `TextView` 的子类。

`TextView` 类有用于记录显示区域及一段文本的字段。它能在区域内自动换行地显示文本，并理解要求滚动文本的消息。它还能把给定坐标对应到文本中的位置，并选择两个位置之间的文本。此外，它还理解要求对选中内容进行替换、剪切、粘贴、重复操作等消息。

因此，`TextView` 与当前的 `ParagraphEditor` 相似。不过，直接面向用户的操作被改造成一个或多个消息，以便它能够通过 `Editor` 与其他视图合作，也能够与不同类型的 `Editor` 配合。

`ActivityText` 必须知道如何获取给定活动的文本。这个操作可以由 `select` 消息触发，通常该消息来自 `Editor`。它还必须能够响应用户通过 `Editor` 发起、与活动有关的消息。

其中有一类消息用于修改视图中展示的数据，这部分由上层类 `TextView` 处理。对 `ActivityText` 本身而言，可能最重要的命令是 `accept`：它要求视图检查当前内容，并把信息提交给网状结构模型，以更新活动属性。

如果把这个类改为表格式视图的子类，而不是当前这种连续文本视图的子类，那么这个视图可以得到很大的改进。

## 示例 4：网状结构图

原文所示网状结构图是 `DiagramView` 类的实例。这是唯一一个无法仅从模型完整推导出视图全部数据的例子：视图必须包含所有符号的形状和位置，因为这些信息并不是模型本身的一部分。（虽然已经有程序能自动排列图中的符号，但我们假定至少仍需要一部分人工调整。）

箭头的连线位置可以由活动之间的依赖关系推导出来；每次绘制时都可以从 `NetworkModel` 获取这些信息。不过，这样做会很慢，因此我们假定 `DiagramView` 为了效率保存了一份相应信息的备份。

与其他视图一样，`DiagramView` 需要提供选择活动所需的两个操作，以及滚动操作——这次需要在两个维度上滚动。

此外，它还需要对视图本身执行一些操作，例如调整图中符号的位置。其他操作则涉及网状结构模型，例如修改活动间的依赖关系，或把活动从一个网状结构转移到另一个网状结构。

## 示例 5：网状结构图的另一种形式

这个视图是前一个例子的替代表示。它使用更小的符号，所以可以在屏幕上展示网状结构中更大的部分。但因为符号太小，不适合把活动名称写在符号内部，用户必须通过其他方式获得名称。

网状结构图显示在一个带网格点的背景上。用户布局图形时可以使用这些网格点；点的位置定义了活动符号允许放置的位置。

该视图可以实现为 DiagramView 的子类，或者让两个图形视图都继承某个共同的视图类。

不过，在当前实现中，两种视图可以由同一个对象交替生成：DiagramView 类有字段 displayType，它可以取 #large 或 #small，各种显示方法根据该字段决定行为。背景网格由一个单独的方法绘制。

### 示例 6：Gantt 图（Gantt Diagram）

这个视图在纵轴上显示各项活动，在横轴上显示时间。在这个特定视图中，每个活动只显示一套时间计划。通过改变阴影，很容易扩展表示方式，以展示每项活动在不影响整体项目进度的前提下有多少时间浮动余地。

该图是 GanttView 类的实例；GanttView 是 ChartView 的子类。ChartView 了解图表背景，例如带文字标记的坐标轴和网格等。它并不了解图表中需要绘制的具体信息——此处就是水平条形。

不过，ChartView 通过提供坐标转换方法，帮助子类展示信息。这些方法把外部使用的坐标系转换为图表显示区域的坐标。

GanttView 能理解用户发送的多种消息，这些消息通常经由 Editor 发出。例如 viewNetwork: 消息提供当前 NetworkModel 的名称。它也能够回答图中给定点下的条形属于哪个活动，并选择与指定活动对应的条形。

此外，它可以把与当前视图有关的网状结构及活动操作继续传给模型。典型操作涉及修改当前进度计划。应当提供的方法包括：对整个网状结构进行向后或向前排程的 backload: 和 frontload:；修改单个活动进度计划的 plannedStart: 和 plannedFinish:；以及让这种修改产生的后果向网状结构前端或后端逐步传播。

NetworkModel 必须能够回答来自 GanttPane 的一些查找，例如返回全部活动列表、整个网状结构的进度计划，以及各个活动的进度计划。

### 示例 7：资源图

这个图以时间为自变量，展示各项活动资源需求之和。（一般而言，每一种资源类型都有一张这样的图。）

该图是 ResourceView 类的实例，它也是 DiagramView 的一个子类。前面对Gantt 图的看法在这里同样适用，不过选择操作需要进一步说明。

视图对 who:point? 的回答，可以是该点横坐标所对应时间上，所有需要资源的活动集合。对 select: activity 消息的响应，可以是突出显示指定活动的资源需求。

为了保持对称，最好把两者结合起来，使所有 NetworkModel 视图都能够进行多项选择。

## 4. Editor（EDITOR）

### 定义

Editor是用户与一个或多个视图之间的接口。它为用户提供合适的命令系统，例如可以随当前上下文动态改变的选项表；同时为各视图提供必要的配合功能和命令消息。

### 说明

用户完成一项任务时，往往同时在屏幕上显示多个视图会更好。用户希望通过指向视图、在选项表中选择，或使用其他方式操作这些视图。选择等命令通常需要同时作用于多个视图。

Editor的目的，就是在屏幕上建立并放置指定的一组视图，配合这些视图，并向用户提供合适的命令界面。

### 示例 8：UserView 工作区

这是一个非常通用的Editor，可以为运行中的 Smalltalk 系统中任何能通过全局变量访问的部分提供用户界面。因此，它也可用来访问规划系统的视图，不过大部分工作都要由用户自己完成。

由于这个Editor始终可用，它被用于启动更专门的Editor，以及执行那些使用频率不足以值得单独制作Editor的命令。例如，把整个数据库输出到文件的命令。

### 示例 9：一个设想中的规划Editor

这个Editor与前一个很相似，但它是在演示网状结构的环境中创建的。因此，针对该网状结构和其中所有活动的消息，都可以直接输入，并通过 `doit` 命令执行。这样一来，网状结构和活动的完整消息共议都可以通过这个Editor访问。

这个Editor并没有以这里所示的独立形式实现，而是作为下一个例子中更大的Editor的一部分实现。

### 示例 10：属于演示Editor的子Editor

这个Editor是更大的演示Editor的一部分（见下一个例子），它就是为在该环境中工作而设计的。（示例 9 的插图实际上就是同一个Editor，只不过滚动到了顶部。）

这个Editor是 `DemoEditor` 类的实例，`DemoEditor` 是 `TextEditor` 的子类。因此，可以在该Editor中输入、保存和修改一般文本。

此外，由于 `DemoEditor` 知道演示网状结构 `anet` 及其所有活动，所以这些对象能理解的任何消息，都可以输入并通过 `doit` 命令执行。

由于这个Editor是更大演示Editor的一部分（见下一个例子），我们还可以执行更一般的表达式，比如 `activity duration`；这里的 `activity` 被解释为当前选中的活动。

这种子Editor的一个变体，可以与 `ListView` 某个子类的实例通信：不允许用户修改所显示的文本，让用户只能执行预先定义的命令，从而限制可执行的操作。

### 示例 11：演示Editor

这个Editor在屏幕上的外观显示于原文下一页。它用于演示：一个给定的模型，可以通过许多不同的视图来展示。

演示Editor实现为 `DemoEditor` 类的实例，它是 `PanedWindow` 的子类。每个窗格都是一个子Editor，与其特定的视图通信。子Editor还会向演示Editor发送命令，并从它那里接收命令。

### 示例 11 插图中的操作文字

- 查找和修改活动（INQUIRE AND EDIT ACTIVITIES）：`activity duration`、`activity persons`、`activity predecessors`、`activity successors`、`activity earlyStart`、`activity lateFinish`。
- 查找和操作网状结构（INQUIRE AND COMMAND NETWORK）：`anet startActivities`、`anet endActivities`、`anet backload: 50`、`anet forwardRank:`。
- 中部和右侧分别展示活动网状结构、Gantt 图及资源需求随时间变化的图。三种视图面向同一个演示网状结构 `anet`。

---

# 第二份文稿（1979 年 12 月 10 日）

## MODELS–VIEWS–CONTROLLERS：模型—视图—控制器

发件人：Trygve Reenskaug

日期：1979 年 12 月 10 日

### 模型（MODELS）

模型表示知识。一个模型可以只是单个对象（那样没什么趣味），也可以是由多个对象组成的某种结构。这里提出的实现方式支持以类似语义网的形式表示知识（如果我正确理解了 Laura 的意思的话）。

一方面是模型及其组成部分，另一方面是模型所有者所感知的、由模型表示的世界；两者之间应当存在一一对应的关系。因此，模型中的节点应当代表问题中一个可以明确识别的部分。

模型中的所有节点都应当处于同一问题层次。把面向问题的节点（例如日历中的约会事项）与实现细节（例如段落）混在一起，会让人不清楚，也被认为是一种不良的设计方式。

### 视图（VIEWS）

视图是其模型的一种（视觉）表示。通常，视图会突出模型的某些属性而隐藏其他属性，因此起着呈现过滤器的作用。

视图与其模型（或模型的一部分）相连接，通过向模型提问来获取呈现所需的数据。视图也可以通过发送适当的消息来更新模型。所有这些提问和消息都必须使用模型自身的术语，因此视图必须理解它所表示的模型属性的语义。（例如，视图可以向模型请求其标识符，并期望得到一个 `Text` 类的实例，但它不能假定模型本身属于 `Text` 类。）

### 控制器（CONTROLLERS）

控制器是用户与系统之间的纽带。它布置相关视图出现在屏幕上的合适位置，向用户提供信息。它也通过选项表或其他供用户输入命令和数据的方式，让用户能够向系统提交信息。控制器接收用户发出的信息，把它转换为适当的消息，再将这些消息传给一个或多个视图。

控制器绝不应当补充视图的绘制工作，例如不应通过在节点视图之间绘制箭头来连接这些视图。

反过来，视图绝不应当知道用户输入的细节，例如Mouse操作和按键。应当始终能够在控制器中编写一个方法，让它向视图发送消息，从而精确地重现任意一系列用户命令。

### Editor（EDITORS）

一个控制器与它的所有视图相连接；这些视图被称为该控制器的组成部分。有些视图会提供一种特殊的控制器——Editor，它允许用户修改视图呈现的信息。这样的Editor可以插入控制器与视图之间的通信路径，并作为控制器的扩展发挥作用。修改一旦完成，Editor就会从路径中移除并被丢弃。

请注意，Editor通过与它相连接的视图所采用的构想与用户交互，因此Editor与这个视图密切相关。控制器应当向视图请求Editor，因为没有其他更合适的来源。

---

## 译者术语说明（非原文）

- `Thing`：这里指用户关注的现实事物或概念，不等于程序中的一个对象。
- `Model`：1979 年 5 月的定义强调“对抽象的主动表示”，而不是简单的数据结构。
- `View`：早期设计中的视图不仅展示数据，也可以执行与自身表示相关的模型操作。
- `Editor` / `Controller`：第一份文稿采用 Editor；第二份文稿改用 Controller，但仍保留一种特殊的临时 Editor。

- `Dependent` / `Observer`：这两份早期文稿没有把后来 Smalltalk-80 的依赖对象通知机制写成 Model 的必备定义；该机制应与后来的实现文献分别阅读。

# 原始英文扫描页（供核对图表与源码）

以下保留原 PDF 的完整页面，包括图表、程序列表、代码注释与历史页码。前面的中文译文引用的图号均可在这些原页中找到。

# The original MVC reports

**Trygve Reenskaug**
**Dept. of Informatics**
**University of Oslo**

I made the first implementation and wrote the original MVC reports while I was a visiting scientist at Xerox Palo Alto Research Laboratory (PARC) in 1978/79. MVC was created as an obvious solution to the general problem of giving users control over their information as seen from multiple perspectives. MVC has created a surprising amount of interest. Some texts even use perverted variants for the opposite purpose of making the computer control the user.

MVC was conceived as a general solution to the problem of users controlling a large and complex data set. The hardest part was to hit upon good names for the different architectural components. Model-View-Editor was the first set. They are described in my first note of May 12, 1979: *THING-MODEL-VIEW-EDITOR -.an Example from a planningsystem*. A scanned version is included below.

After long discussions, particularly with Adele Goldberg, I ended with the terms Model-View-Controller as described in my second note of December 10, 1979: *MODELS - VIEWS – CONTROLLERS.* A scanned version is included at the end of this document.

Jim Althoff and others implemented a version of MVC for the Smalltalk-80 class library after I had left Xerox PARC; I was in not involved in this work.

Oslo, February 12, 2007
Trygve Reenskaug

# THING-MODEL-VIEW-EDITOR
## *an Example from a planningsystem*

To LRG

From Trygve Reenskaug

Filed on [IVY]<Reenskaug>SMALL> TERMIN0LOGY2.DOC

Date 12 MAY 1979

The purpose of this note is to explore the *thing-model-view-editor* metaphors through a coherent set of examples. The examples are all drawn from my planningsystem, and illustrate the above four notions. All examples have been implemented, albeit not within the clean class structure described here. The metaphors correspond to *real world-Model-view-Tool* as proposed in *A note on DynaBook requirements* ([Ivy]<Reenskaug>DynaBook.doc).

## THING

### *DESCRIPTION OF TERM*

*Something that is of interest to the user. It could be concrete, like a house or an integrated circuit. It could be abstract, like a new idea or opinions about a paper. It could be a whole, like a computer, or a part, like a circuit element.*

### *EXAMPLE: A LARGE PROJECT*

The Thing is here a large project. It could be the design and construction of a major bridge, a power station or an off-shore oil production platform.

Such projects represent complex tasks with a very large number of interdependent details. Those responsible for a project have to keep track of all these details and their dependencies in order to understand the consequences of various real or proposed situations that may arise.

A great variety of different abstractions are used as aids in controlling large projects. Each abstraction highlights particular aspects of the total project, and are used alone or together with other abstractions for the control of those aspects. Examples of abstractions are ways of thinking about the requirements for materials; cost estimating; and various budgeting and accounting theories.

One particularly useful abstraction is the notion of *activity networks* or *Pert diagrams.* This abstraction links together *what* should be done with *who* should do it and *when* it is to be done.

In the network abstraction, every task that has to be performed in the project is mapped into a simple element called an *activity.* Basically, an activity is characterised by its *duration* and its *predecessors.* The duration specifics the time needed to perform the corresponding task, and

the predecessors are the activities that have to be completed before the present activity can be started.

This simple abstraction is usually extended in a nummber of ways. Based upon the activities' predecessors, it is easy to find the successors of each activity. The *start activities* of the network are all activities that have no predecessors, and the *end activities* are the activities that have no successors. Other information may be tacked on to each activity. The activity's resource requirements is one example, the cost and cash flow associated with the activity is another.

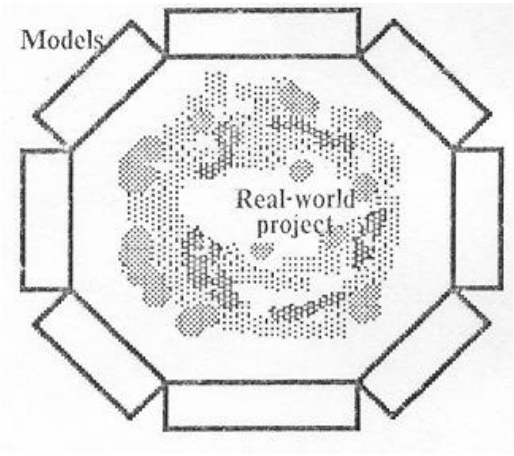
## MODEL

### *DEFINITION*

*A Model is an active representation of an abstraction in the form of data in a computing system*

### *COMMENTS*

As mentioned above, there are in general many different ways of abstracting the same Thing, and it is therefore often useful with several, coexisting Models of a given 'Thing. Alternatively, one may think of the project as having one, large Model that is subdivided into a number of sub-Models.



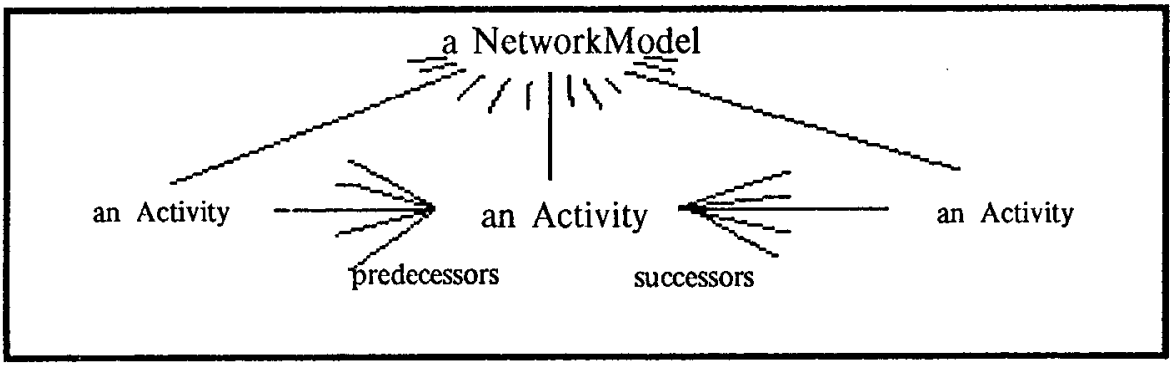

The models are represented in the computer as a collection of data together with the methods necessary to process these data.

Ideally, all models should be totally consistent. This ideal is not attainable in practice because it would require an overwhelming bureaucracy and stifling rigidity. One should therefore accept some inconsistencies, the aim being that the total set of models should be a reasonabely accurate representation of the project without spending too much effort on trivialities.

In our example, a given network Model is represented in the Smalltalk system as an instance of class NetworkModel. Each activity in the network Model is represented as an instance of class Activity.

One of the fields defined for class NetworkModel is **activities**, a Dictionary connecting activity names (a UniqueString) and instances of Activity. Three of the fields defined for class Activity are **network**, **predecessors** and **successors**. **network** contains a pointer to the relevant instance of NetworkModel, and **predecessors** and **successors** are Vectors of pointers to instances of Activity.

The overall structure of our Model (= the Smalltalk representation of a network model of a project) is thus as follows: .




Within this general framework, we may now add information about our project that may be connected to the network as a whole, and to each of its activities. To the network, we may for example add the fields **plannedStart** and **plannedFinish**. To the definition of class Activity we may add the fields **duration**, **earlyStart**, **lateStart**, and **resourccRcquirements**. We may also add fields for late start and late finish, but decide to compute these quantities whenever needed. (From **duration** and **earlyStart** or **lateStart**).

Note that the network Model with its sub-Models for each activity contains no information about how the information may be shown on the screen, it is a clean representation of the abstract project model.

## VIEW

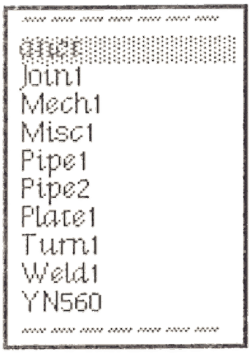
***DEFINITION***

*To any given Model there is attached one or more Views, each View being capable of showing one or more pictorial representations of the Model on the screen and on hardcopy. A View is also able to perform such operations upon the Model that is reasonabely associated with that View.*

***COMMENT***

The implementation of all Views would be simplified if they were based upon the Form-Path-Image metaphors.

**EXAMPLE 1: List of networks**




This View belongs to a super-network, i.e. a collection of networks. It is thus slightly outside the scope of the Models discussed so far, but is included for completeness. The appearance of a list of networks is shown to the left.

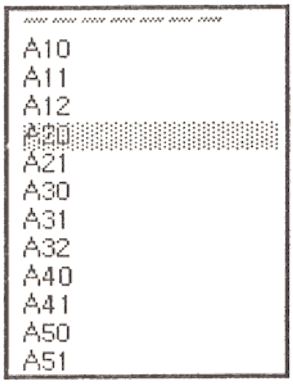
A list of networks is an instance of class NetworkList, which is a subclass of ListView.

A ListView has fields where it remembers its frame, a list of textual items and a possible selection within the list. It is able to display its list on the screen within its frame, and reacts to messages asking it to scroll itself. It understands a message asking it for an item that is positioned at a given point within its frame, and a message asking it to select a given item.

A ListView is thus somewhat similar to a ListPane in the present system, but it is not a subclass of Window and cannot be scheduled. It must therefore depend upon an Editor to tell it where its frame is and to arrange for scrolling. One possible sequence for selection is that the Editor reacts to redbug and asks the ListView for the item that is pointed to. The Editor then asks the ListView and any other Views to select that item. This separation of user interface and actual command execution provides for great flexibility in the Editor.

The class NetworkList builds on the general facilities of the ListView. It must know that the list in question is a list of networks, and it must know how to hold of such a list. Further, it must be able to execute commands that might be of interest in connection with its View. We will only consider two such commands: give the name of selected network, and tell the Editor to edit a given network.

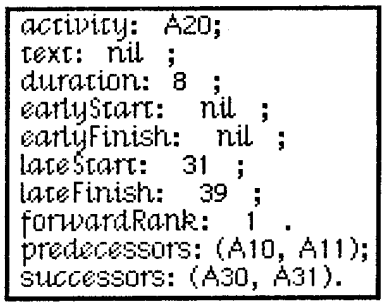
***EXAMPLE 2: List of activities within a network***




A list of activities within a network is an instance of class ActivityList, which is a subclass of ListView (see above for description of this general class).

An instance of lass ActivityList must know which network it belongs to, and how to get hold of a list of all the activities in that network. 'There are no special commands other than the general selection mechanisms provided by ListView.

***EXAMPLE 3: View of activity attributes***




This is a textual presentation of all the attributes of an activity. It is an instance of class ActivityText, which is a subclass of TextView.

Class TextView has fields where it remembers its frame and a paragraph of text. It is able to display the text within its frame with wrap-around, and understands messages that ask it to scroll the text within the frame. It is also able to link given coordinates to positions within the text, and to select text between given positions. It further understands messages asking it to perform various manipulations based upon the selection like *replace*, *cut*, paste, *again*, etc.

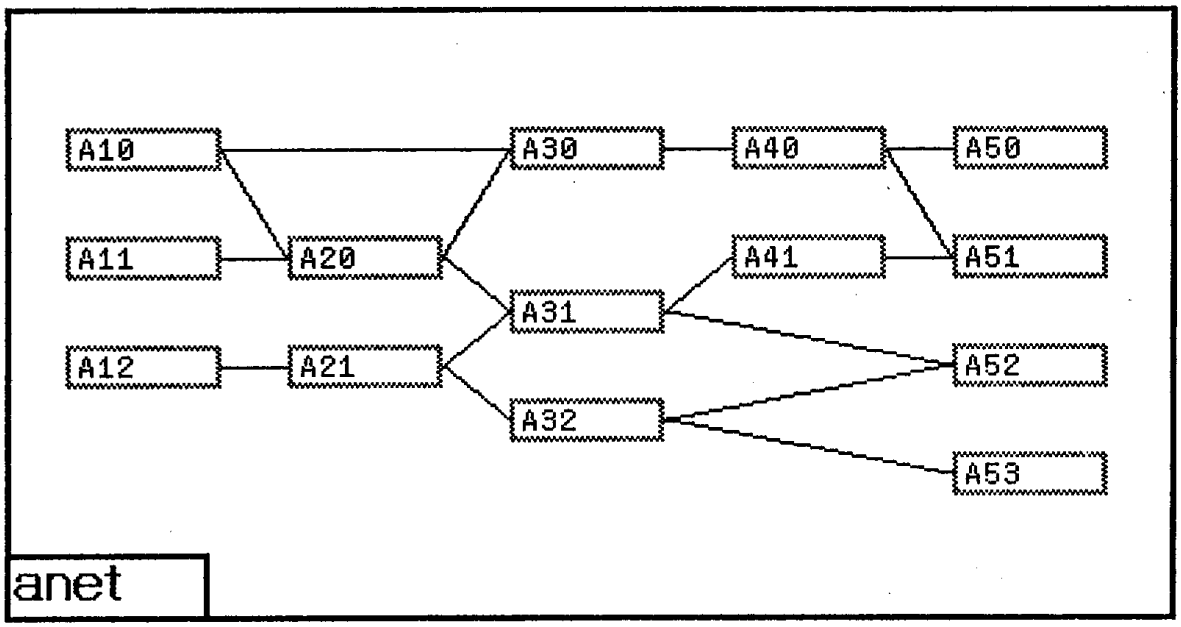
Class TextView is thus similar to the present ParagraphEditor, but each direct user interface is changed into one or more messages so that it may work together with other Views through an Editor, and with a variety of different Editors.

Class ActivityText must know how to get hold of the text for a given activity. This operation may usefully be triggered by a *select* message, usually from an Editor. It must further be able

to react to messages about the activity that are initiated by the user through an Editor. One set of such messages apply to modification of the data presented in the View, this is taken care of by the superclass TextView. The probably most important command to ActivityText itself is *accept*, requsting the View to check its present contents and to pass the information on to the network Model as an update of the attributes of the activity.

This View could be greatly improved if the class was made subclass of a tabular View rather than the present running text

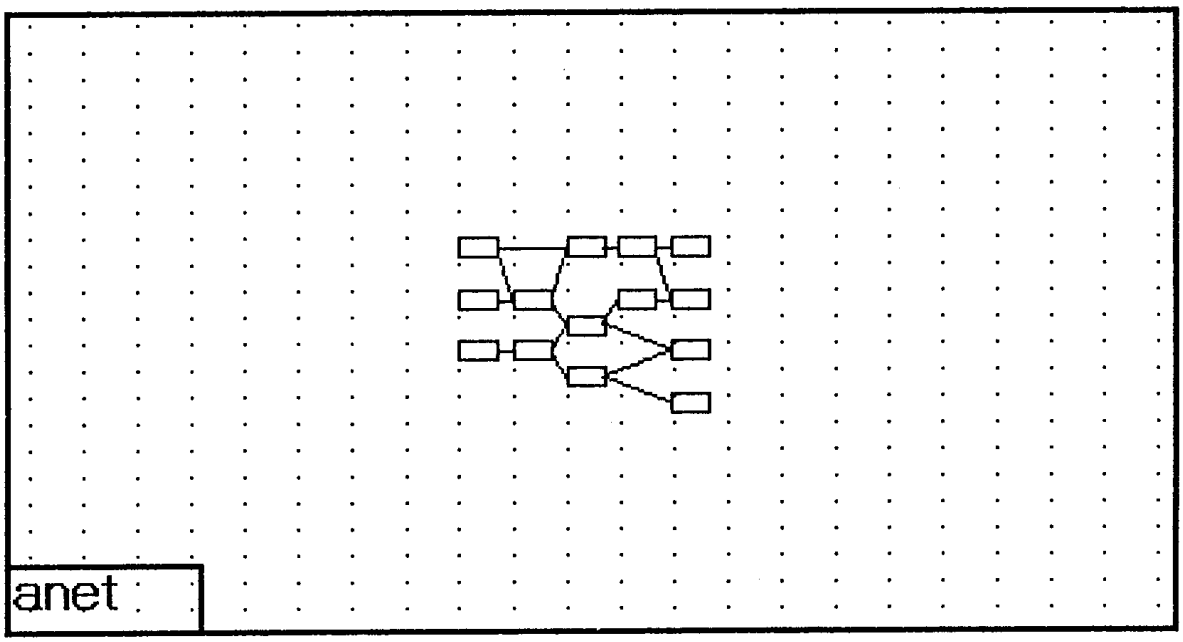
***EXAMPLE 4: Network diagram***




The diagram above is an instance of class DiagramView. It is the only example where the data in the View cannot be fully deduced from its Model, and it must have fields containing the shape and position of all the symbols, since this information is not part of the Model itself. (Programs exist for the automatic positioning of symbols within a diagram. but we assume that at least some manual editing will be required). The placement of the arrows may be deduced from the dependencies between the activitities, this information may be fetched from the NetworkModel whenever needed for the display. Such a procedure will be very slow, however, and we assume that the Diagram View keeps a copy of that information for efficiency reasons.

Just as all the other views, a Diagram View will need to supply the two operations needed to select an activity, and the operations needed for scrolling (this time in two dimensions). In addition, it will need some operations on the View itself, they have to do with the positioning of the symbols in the diagram. Other operations have to do with network Model, for example *modify dependencies* of an activity and *transfer activity* from one network to another.

***EXAMPLE 5: A variation on the network diagram***

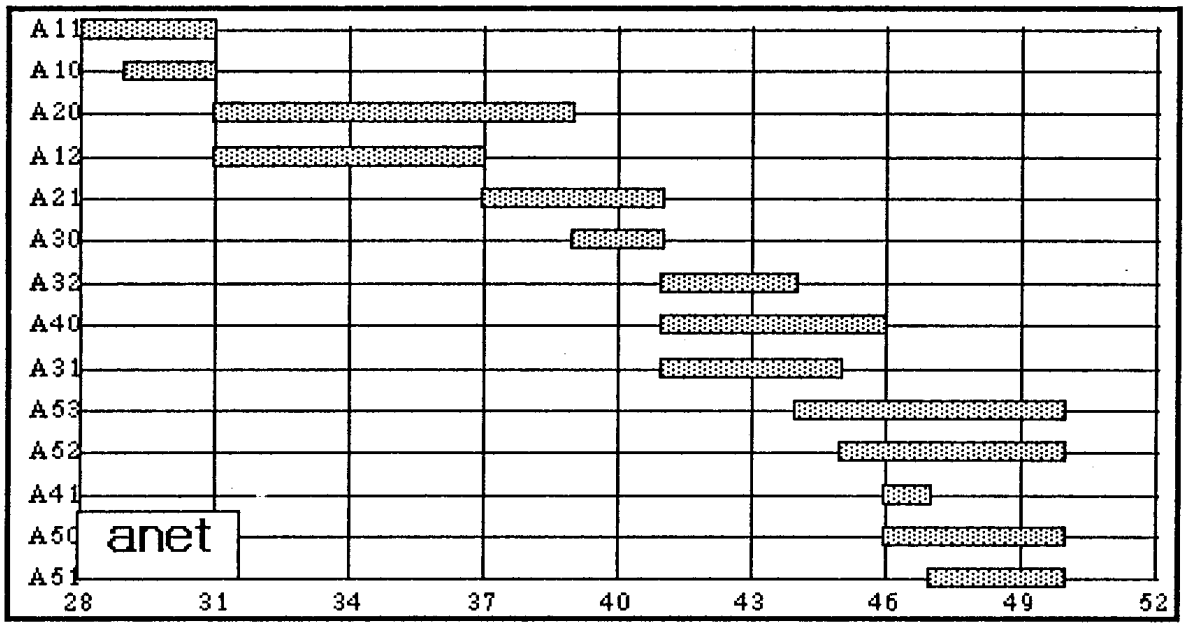



This View provides an alternative to the previous example. The symbols are smaller, and it therefore possible to present a larger part of the network on the screen. The small symbol size makes it impractical to print the activity name in the symbol, and the user must get this information by some other means.

The network diagram is presented on a gridded background. This is used when the user is laying out the diagram, and the dots define the permissible positions of the activity symbols.

This View could be implemented as a subclass of DiagramView, or both could be subclasses of some common View. In the present implementation, however, both Views may be generated alternately by the same object: class Diagram View has has a field displayType that may take the values #large or #small, and the various display methods are controlled by that field. The gridding is displayed in the diagram through a separate method.

***EXAMPLE 6: Gantt diagram***



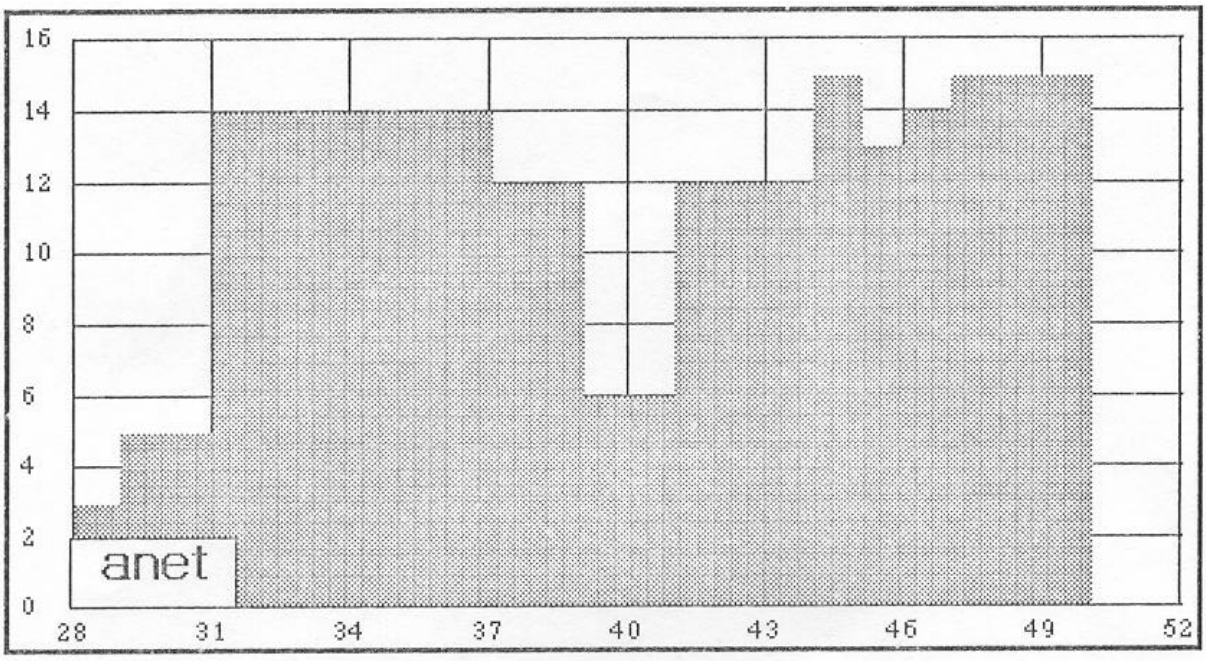

This View shows the activities along the vertical axis and time along the horizontal axis. In this particular view, only one time schedule for each activity is shown. Through varying the shading, it is quite easy to extend the presentation to include how much each activity may *float* in time without upsetting the overall project schedule.

The diagram is an instance of class GanttView, which is a subclass of ChartView. ChartView knows about the diagram background: axis with legend, gridding etc. It does not know anything about the information to be put into the diagram, in this case the horizontal bars. It aids its subclasses in presenting this information, however, through providing methods for conversion from whatever coordinate system is used externally to the frame coordinates of the diagram.

Class GanttView understands a number of messages from the user, most commonly given through an Editor. One such message is **viewNetwork:**, this message provides the name of the current NetworkModel. It will also be able to answer a question about which activity belongs to the bar under a given point in the diagram, and to select the bar belonging to a given activity. It is also able to pass on operations on the network and its activities that are related to this particular View. Typical operations have to do with modifying the current schedule. Operations should be provided for planning the network as a whole (**backload:** and **frontload:**), for modifying the schedule of a single activity (**plannedStart:** and **planncdFinish:**), and for letting the consequences of such a modification work its way towards the front or back end of the network.

The NetworkModel must be able to answer a few questions rom a GanttPane giving a list of all activities, the schedule of the network as a whole, and the schedule of each individual activity.

***EXAMPLE 7: Resource diagram***




This diagram shows the sum of the resource requirements for the activities as a function of time. (In general, there will be one such diagram for each resource type.)

The diagram is an instance of class ResourceView, which is another subclass of DiagramView. The comments about the Gantt-diagram apply to this one as well, but the concept of selection needs elaboration. The views answer to ***who:****point?* could be the set of all activities that need resources at the time cooresponding to the point's x-value. The reaction to the message *select: activity* could be to highlight the resource requirements of that particular activity. For symmetry, it would be nice to combine the two, and open the possibility for multiple selections in all NetworkModel Views.

## EDITOR

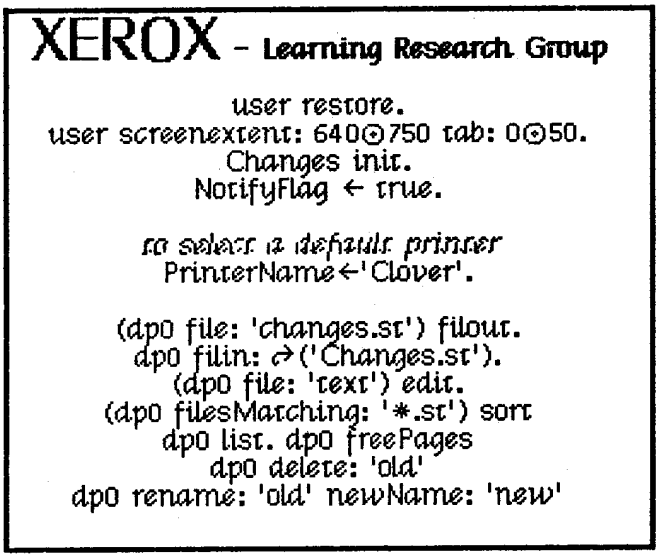
***DEFINITION***

*An Editor is an interface between a user and one or more views. It provides the user with a suitable command system, for example in the form of menus that may change dynamically according to the current context. It provides the Views with the necessary coordination and command messages.*

***COMMENT***

A user task may commonly be better performed with several Views present on the screen at the same time. The user will want to manipulate these Views through pointing at them, through selection in menus, or through some other means. Commands like selection typically apply to several Views at the same time. The purpose of an Editor is to establish and position a given set of Views on the screen, to coordinate them and to provide the user with a suitable command interface.

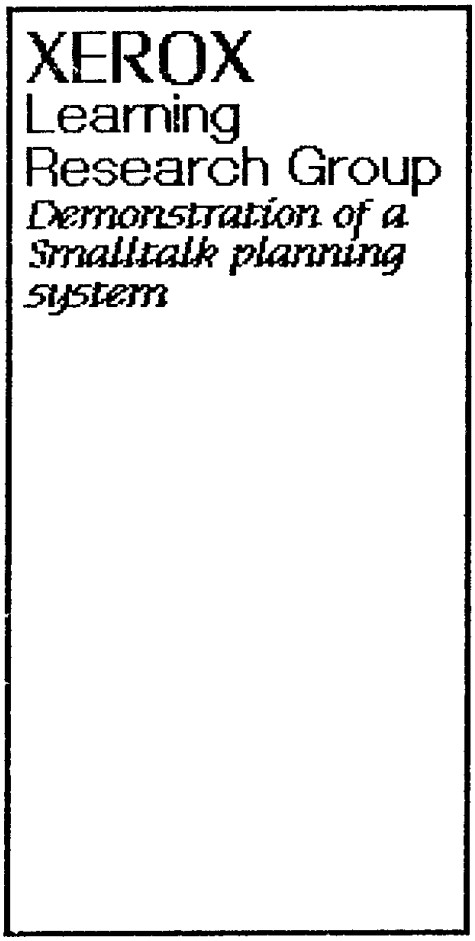
***EXAMPLE 8: UserView workspace***




This is a very general Editor, providing a user interface to any part of the running Smal1talk system that can be reached from a global variable. It can therefore be used as an interface to the Views of the planningsystem, but it leaves most of the work to the user.

Since this Editor is always available, it is used to start up more specialized Editors and to perform commands that are not used sufficiently frequently to warrant such editor. One example of the latter is a command for the fileout of the complete data base.

***EXAMPLE 9: An imaginary planning Editor***

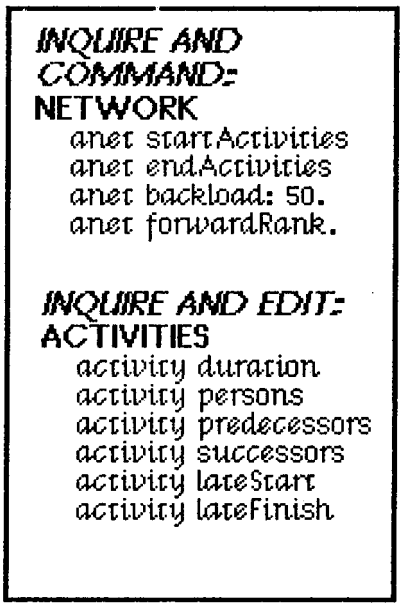



This Editor is very similar to the previous one, but it has been created in the environment of a demonstration network. Messages to that network and all its activities may therefore be typed in and executed directly through the *doit* command. The complete protocols of the network and the activities are therefore available through this Editor.

The Editor has not been implemented in this form, but as a part of a larger Editor shown in the next example.

***EXAMPLE 10: A sub-Editor belonging to the***

***demonstration Editor.***




This Editor is part of the larger *demonstration Editor* (see next example), and is designed to work within its context. (The illustration of example 9 was actually the same Editor scrolled to its top.)

The Editor is an instance of class DemoEditor, which is a subclass of TextEditor. General texts may therefore be typed in, stored and edited in this Editor. Further, since DemoEditor knows about the demonstration network *anet* and all its activities, any message understood by these objects may be typed in and executed through a *doit* command.

Since this Editor is part of the larger *demonstration Editor* (see next example), we may execute the more general expressions like *activity duration*, where *activity* is interpreted as the currently selected activity.

A variation on this sub-Editor could talk to a an instance of a subclass of ListView, limiting the user's possibilities to executing predefined commands through prohibiting any editing of the text shown.

***EXAMPLE 11: A demonstration Editor.***

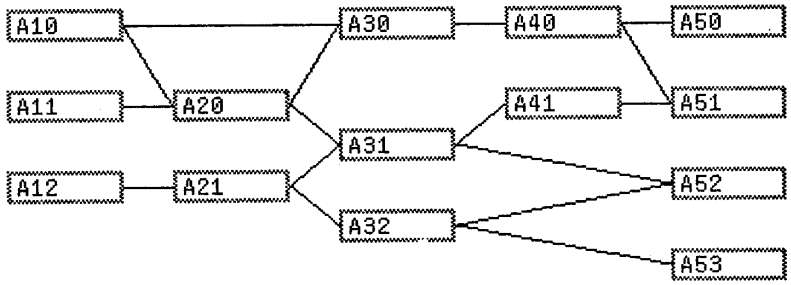
The appearance of this Editor on the screen is shown on the next page. The Editor is used to demonstrate that one given Model may be displayed through many different Views.

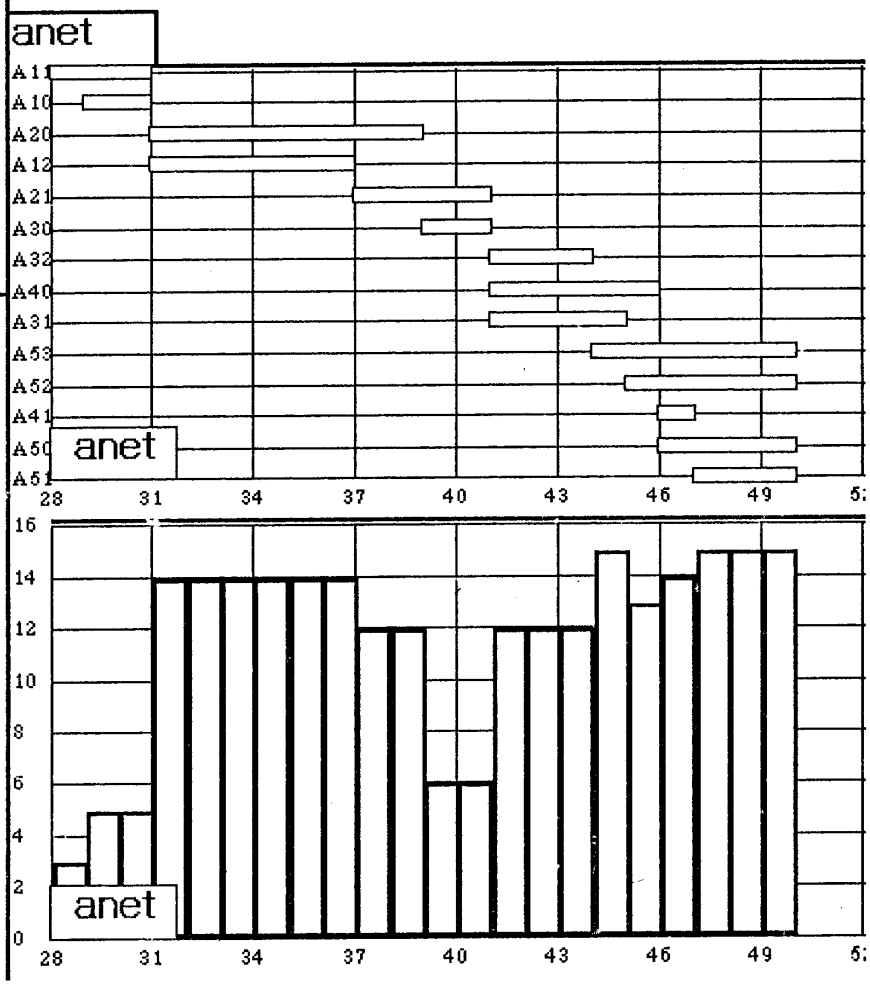
The demonstration Editor is implemented as an instance of class DemoEditor, which is a subclass of PanedWindow. Each of its panes is a sub-Editor that communicates with its particular View. It also gives commands to the demonstration Editor and receives commands from it.

***INQUIRE AND EDIT:***
**ACTIVITIES**
activity duration
activity persons
activity predecessors
activity successors
activity lateStart
activity lateFinish

***INQUIRE AND COMMAND:***
**NETWORK**
anet startActivities
anet endActivities
anet backload: 50.
anet forwardRank.

A10
A11
A12
A20
A21
A30
A31
A32
A40
A41
A50
A51
A52
A53



From: Trygve Reenskaug

Date: 10 December 1979

# MODELS - VIEWS - CONTROLLERS

## MODELS

Models represent knowledge. A model could be a single object (rather uninteresting), or it could be some structure of objects. ~~The proposed implementation supports knowledge represented in something resembling *semantic nets* (If I understand Laura correctly)~~

There should be a one-to-one correspondence between the model and its parts on the one hand, and the represented world as perceived by the owner of the model on the other hand. The nodes of a model should therefore represent an identifiable part of the problem.

The nodes of a model should all be on the same problem level, it is confusing and considered bad form to mix problem-oriented nodes (e.g. calendar appointments) with implementation details (e.g. paragraphs).

## VIEWS

A view is a (visual) representation of its model. It would ordinarily highlight certain attributes of the model and suppress others. It is thus acting as a *presentation filter*.

A view is attached to its model (or model part) and gets the data necessary for the presentation from the model by asking questions. It may also update the model by sending appropriate messages. All these questions and messages have to be in the terminology of the model, the view will therefore have to know the semantics of the attributes of the model it represents. (It may, for example, ask for the model's identifier and expect an instance of Text, it may not assume that the model is of class Text.)

## CONTROLLERS

A controller is the link between a user and the system. It provides the user with input by arranging for relevant views to present themselves in appropriate places on the screen. It provides means for user output by presenting the user with menus or other means of giving commands and data. The controller receives such user output, translates it into the appropriate messages and pass these messages on .to one or more of the views.

A controller should never supplement the views, it should for example never connect the views of nodes by drawing arrows between them.

Conversely, a view should never know about user input, such as mouse operations and keystrokes. It should always be possible to write a method in a controller that sends messages to views which exactly reproduce any sequence of user commands.

## EDITORS

A controller is connected to all its views, they are called the parts of the controller. Some views provide a special controller, an *editor*, that permits the user to modify the information that is presented by the view. Such editors may be spliced into the path between the controller and its view, and will act as an extension of the controller. Once the editing process is completed, the editor is removed from the path and discarded.

Note that an editor communicates with the user through the metaphors of the connected view, the editor is therefore closely associated with the view. A controller will get hold of an editor by asking the view for it - there is no other appropriate source.